

Compositions of Variant Experts for Integrating Short-Term and Long-Term Preferences

JAIME HIEU DO, Singapore Management University, Singapore

TRUNG-HOANG LE, Faculty of Information Technology, University of Science, VNU-HCM, Ho Chi Minh City, Vietnam and Vietnam National University, Ho Chi Minh City, Vietnam

HADY W. LAUW, Singapore Management University, Singapore

In the online digital realm, recommendation systems are ubiquitous and play a crucial role in enhancing user experience. These systems leverage user preferences to provide personalized recommendations, thereby helping users navigate through the paradox of choice. This work focuses on personalized sequential recommendation, where the system considers not only a user's immediate, evolving session context, but also their cumulative historical behavior to provide highly relevant and timely recommendations. Through an empirical study conducted on diverse real-world datasets, we have observed and quantified the existence and impact of both short-term (immediate and transient) and long-term (enduring and stable) preferences on users' historical interactions. Building on these insights, we propose a framework that combines short- and long-term preferences to enhance recommendation performance, namely Compositions of Variant Experts (CoVE). This novel framework dynamically integrates short- and long-term preferences through the use of different specialized recommendation models (i.e., experts). Extensive experiments showcase the effectiveness of the proposed methods and ablation studies further investigate the impact of variant expert types.

Additional Key Words and Phrases: short-term preference, long-term preference, variant experts, compositions

ACM Reference Format:

Jaime Hieu Do, Trung-Hoang Le, and Hady W. Lauw. 2025. Compositions of Variant Experts for Integrating Short-Term and Long-Term Preferences. 1, 1 (June 2025), 25 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Recommendation systems have become an integral part of the online digital realm, significantly enhancing user experience by providing personalized suggestions. Leveraging user preferences from past interactions, these systems help users navigate through the vast array of available choices faster by providing relevant recommendation. Modern recommendation systems, particularly sequential recommendation, consider the user's temporal preferences, including recent interactions such as clicks, views, purchases, and location check-ins, to provide recommendations.

There is a distinction between *session-based* and *session-aware* recommendation systems. Session-based recommenders focus on capturing sequential user behavior within a single session, reflecting a user's short-term preference and aiming to predict the next item in that sequence. Conversely, session-aware recommenders incorporate both the current

Authors' Contact Information: [Jaime Hieu Do](mailto:dinhhieudo.2020@smu.edu.sg), dinhhieudo.2020@smu.edu.sg, Singapore Management University, Singapore; [Trung-Hoang Le](mailto:thle.2017@smu.edu.sg), thle.2017@smu.edu.sg, Faculty of Information Technology, University of Science, VNU-HCM, Ho Chi Minh City, Vietnam and Vietnam National University, Ho Chi Minh City, Vietnam; [Hady W. Lauw](mailto:hadywlauw@smu.edu.sg), hadywlauw@smu.edu.sg, Singapore Management University, Singapore.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

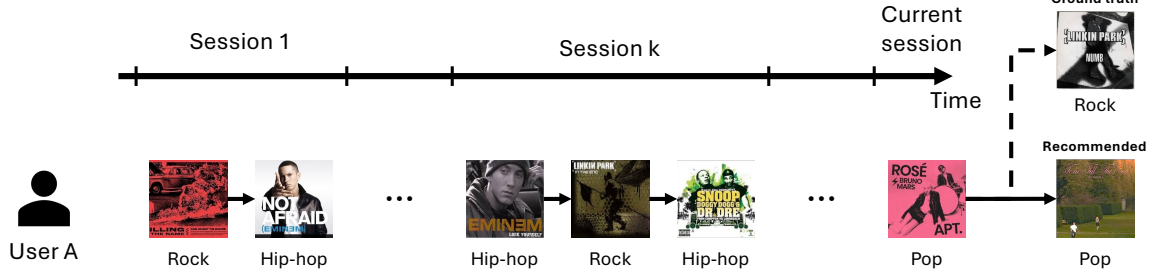


Fig. 1. User A might want to be recommended a rock song based on their long-term interest in rock and hiphop, rather than a pop song based on their short-term interest in a trending pop song.

session context and the user’s historical behavior, thereby encompassing their long-term preference. This may enable session-aware recommendation systems to offer more accurate personalized recommendations.

A critical aspect of session-aware recommendation systems is the differentiation between short- and long-term user preferences. The former are derived from the user’s recent interactions within the current session, while the latter are based on historical behavior over a more extended period. An illustrative example is shown in Figure 1, user A is on a streaming music platform. Her historical sessions show a strong preference for rock and hip-hop, with representative artists such as “*Rage Against the Machine*”, “*Eminem*”, and “*Linkin Park*”. However, in the current session, however, she is listening to “*APT.*” by “*Bruno Mars and Rosé*”, a trending pop song, indicating a short-term preference for pop. The challenge is deciding whether to recommend more pop songs or stick with user A’s usual hip-hop/rock. If only short-term preferences are considered, the system may recommend more pop songs, which may not align with her typical tastes. Conversely, focusing solely on long-term preferences may overlook her current interest. Understanding and effectively exploiting these preferences can lead to more accurate and personalized recommendations.

Integrating short-term and long-term preferences poses several challenges. Existing session-based recommendation models often focus on capturing short-term preferences, neglecting long-term preferences. On the other hand, traditional recommendation models that capture long-term preferences may not be suitable for session-aware recommendations due to the lack of ability to model sequential user behavior. To address this gap, we propose compositions of variant experts (CoVE), a novel framework that integrates both short- and long-term preferences to enhance recommendation performance by leveraging the strength of multiple types of experts working in concert.

Contribution. First, we explore the existence and impact of short- versus long-term preferences in session-aware recommendation systems in section 2. Second, we propose the Compositions of Variant Experts (CoVE) with two main variants, which dynamically integrate short- and long-term preferences in section 3. Third, in section 5, we conduct extensive experiments to demonstrate the effectiveness of the proposed models and analyze the impact of the variant expert types, the number of experts, as well as the gating mechanism.

2 Short-Term and Long-Term Preferences in Session-Aware Recommendation

Let $\mathcal{P}$ be the universal set of items, where $N = |\mathcal{P}|$ is the total number of items. Let $\mathcal{U}$ be the set of users, $M = |\mathcal{U}|$ is the total number of users. A user $u \in \mathcal{U}$ interacts with several items within a short span of time, denoted t , forming a session $S_t^u = \langle p_{t,1}, p_{t,2}, \dots, p_{t,K} \rangle$, where $p_{t,k} \in \mathcal{P}, \forall k \in \{1, 2, \dots, K\}$, K is the total number of items in session S_t^u . All of these user interactions from multiple sessions in chronological order $C_{1:T}^u = \langle S_1^u, S_2^u, \dots, S_T^u \rangle$ reflect both user *short-term* within a session and *long-term* preferences which are items across all the sessions. We presuppose the most recent

Table 1. Notations

Symbol	Description
$\mathcal{P} = \{1, \dots, p, \dots, N\}$	the set of N items, $N = \mathcal{P} $
$\mathcal{U} = \{1, \dots, u, \dots, M\}$	the set of M users, $M = \mathcal{U} $
$S_t^u = \langle p_{t,1}, p_{t,2}, \dots, p_{t,K} \rangle$	user u 's session at time period t with a total of K items
$C_{1:T}^u = \langle S_1^u, S_2^u, \dots, S_T^u \rangle$	set of user u 's sessions from time period 1 up to time period T
$\{f_1, f_i, \dots, f_n\}$	the set of n experts
$f(u, C_{1:T}^u) \rightarrow \mathbb{R}^N$	predicted scores of N items
h_t^u	user u representation up to time t
$\Psi(p)$	item p representation
$\beta(p)$	item p bias
$g(p_{m,t})$	n -dimensional output of the gating function w.r.t. item input $p_{m,t}$
$g(p_{m,t})_i$	i -th component of $g(p_{m,t})$

session S_T^u of $C_{1:T}^u$ is the current browsing session of user u . The main task is to predict the next probable item $p_{T,K+1}$, where K is the total number of items in the current session S_T^u .

Let f be an “expert”, in this case, a model, defined as a function $f(u, C_{1:T}^u) \rightarrow \mathbb{R}^N$ that maps an input set of sessions from user u , $C_{1:T}^u$, to an N -dimensional vector of real values, where each value denotes the predicted ranking score for an item. In this article, we use the terms “expert” and “preference model” interchangeably.

Here, we discuss the categorization of f into two, i.e., short-term expert and long-term expert. The following properties are useful in determining to which category an expert belongs.

Property 1 (SINGLE SESSION PROPERTY) The model processes only one session at a time, without any cross-session interactions. Specifically, when provided with multiple sessions, the model considers only the most recent one, i.e., $f(u, C_{1:T}^u) = f(u, S_T^u)$.

Property 2 (USER INDEPENDENCE PROPERTY) Given an identical sequence of input items, the model, denoted as f , always produces the same output regardless of user-specific information. Formally, for any two users $u \neq u'$, $f(u, C_{i:j}^u) = f(u', C_{i':j'}^{u'}), \forall C_{i:j}^u = C_{i':j'}^{u'}, i \leq j, i' \leq j'$.

Based on these two properties, short-term and long-term preference model are formally defined as follows:

Definition 2.1. (SHORT-TERM PREFERENCE MODEL) A model is considered as a short-term preference model if both **Property 1** and **Property 2** hold. In particular, a short-term preference model $f(u, C_{1:T}^u)$ takes as input a single sequence of items S_T^u (following **Property 1**) and aims to predict the next probable item $p_{T,K+1}$. The model is user independent, meaning for the same input sequence, it always produces the same output regardless of the user, i.e., $f(u, S_T^u) = f(u', S_T^{u'}), \forall S_T^u = S_T^{u'}, u \neq u'$.

Definition 2.2. (LONG-TERM PREFERENCE MODEL) A model is considered as a long-term preference model if either **Property 1** or **Property 2** does not hold. A long-term preference model $f(u, C_{1:T}^u)$ considers historical sessions $C_{1:T}^u$ or user u , or both as inputs to predict the most probable item $p_{T,K+1}$. The long-term model is user-dependent, meaning for two users u and u' , $u \neq u'$, the model may produce different outputs given the same historical sessions, i.e., $\exists f(u, C) = f(u', C)$. Additionally, a model that considers more than one session is also a long-term preference model. The long-term preference model may learn user underlying preferences, concentrated into a dense representation h^u or model the inter-session relations instead of one single session.

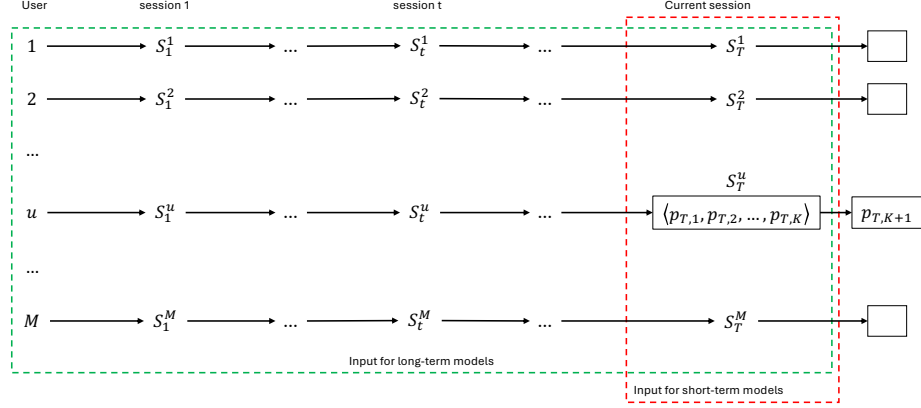


Fig. 2. An illustrative example distinguishing inputs for long-term and short-term preference models.

A short-term preference model operates solely on the interactions occurring within the current session, without incorporating any external information such as user demographics, historical preferences, or side data. This makes it well-suited for scenarios where long-term user data is unavailable or irrelevant. A notable example of a short-term preference model is GRU4REC [22], which utilizes Gated Recurrent Unit (GRU) [10], widely used to model sequential data such as text. GRU4REC processes a sequence of recently interacted items to generate recommendations, focusing exclusively on short-term engagement patterns rather than long-term user history. For better illustration, we highlight the inputs of short-term preference model in the red box in Figure 2.

A long-term preference model leverages stored user information—such as user representation or historical interactions—to enhance future recommendation predictions. It could vary from general collaborative filtering to sequential models which utilize previous sessions together with the current one. For instance, Bayesian Personalized Ranking (BPR) [52] takes a user as input to produce a personalized list of recommended items (green box in Figure 2). Another example instance of long-term preference model, HGRU4Rec [49], uses an additional GRU layer to model all previous sessions of the same user into a condensed representation, which is combined with the current session for next-item prediction.

To further verify the existence of short-term and long-term preferences in session-aware recommendation systems, we conduct an empirical analysis on the public Diginetica¹ and RetailRocket² datasets. Here, we particularly analyze the recommendation performance of two candidate models: one that only models long-term preferences without short-term preferences, and the other vice versa. In particular, we use two representative models, GRU4REC [22], which models short-term preferences through sequential session interactions, and BPR [52], which captures long-term user preferences through user-item interactions. More details on the datasets are provided in subsection 5.1. In these datasets, user interactions occur in sessions, where each session consists of a sequence of item interactions within a given time window, and users may have multiple sessions over time. We hypothesize that there exist both short-term and long-term preferences in session-aware recommendations. To validate this hypothesis, we evaluate the recommendation effectiveness of BPR and GRU4REC using the same training data but formatted to suit each model’s input requirements. BPR processes a user-item interaction matrix without sequential information, while GRU4REC takes a sequence of recently interacted items as input. Both models generate ranked recommendation lists as next item prediction. To

¹<https://competitions.codalab.org/competitions/11161>

²<https://www.kaggle.com/datasets/retailrocket/e-commerce-dataset>

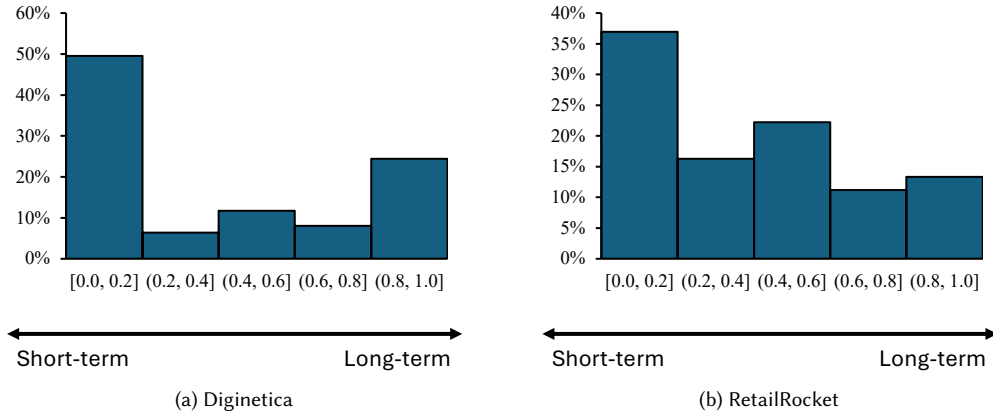


Fig. 3. Histogram of user preference values

determine whether an item aligns more with short-term or long-term preferences, we compare the ranking of the ground truth item in the test set. If BPR ranks the item higher than GRU4Rec, we classify it as a long-term preference (bit = 1). Conversely, if GRU4Rec ranks it higher, we classify it as a short-term preference (bit = 0). In cases where both models rank the item equally, we assign a value of 0.5.

The histograms in Figure 3 visualize the distribution of long-term bits (as a fraction) across the test datasets. While some users have extreme long-term preferences, with an average long-term bit value of 1, others do not have any long-term interest with a value of 0. The average values are 0.39 and 0.38 for Diginetica and RetailRocket datasets respectively, indicating that short-term preferences are slightly more dominant.

Through this empirical analysis, we observe that there exists both short-term and long-term preferences in personalized sequential recommendation, which can drive user behavior differently given their personal preferences. Effectively modeling and balancing these factors hold promise for enhancing recommendation quality, thereby improving user engagement.

3 Compositions of Variant Experts (CoVE)

In this section, we discuss the use of variant experts for session-aware recommendations. The core idea is to leverage different types of both short-term and long-term preference models as experts, each providing unique insights into the data, that collectively combine the strengths of different experts to create a more effective recommendation model.

3.1 Preliminary

Here, we discuss background knowledge of well-known Mixture-of-Experts methods including continuous mixture of experts and sparse mixture of experts.

Mixture-of-Experts. (MoE) model [25] is designed to increase the model’s capacity while maintaining low computational costs by using multiple “experts”. This gating mechanism allocates input tokens to different experts, making each expert specified with distinct input data. There are several ways to implement the gating network $g(x)$ [11, 15, 74], but a simple and commonly used approach is to apply a softmax function over the logits produced by a linear layer [54]:

$$g(x) := \text{Softmax}(x \cdot W_g) \quad (1)$$

where W_g represents the learnable parameters of the linear layer and Softmax formula is given as follows:

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (2)$$

Continuous Mixtures of Experts. Mathematically, the continuous MoE model can be formulated as:

$$\sum_{i=1}^n g(x)_i f_i(x) \quad (3)$$

where $g(x)_i$ is the i -th component in the n -dimensional outputs, for the i -th expert $f_i(x)$. Here, each expert f_i receives the same input x but is weighted differently based on the gating output. The learnable gating function dynamically determines the contribution of each expert, assigning higher weights to more relevant experts. In this continuous MoE framework, all experts participate in the final aggregation, with their influence proportional to their assigned weights.

Sparse Mixtures of Experts (SMoE) [54]. SMoE is a variant of MoE that selects only the top-K experts based on the gating function, reducing computational costs by routing the input through a subset of experts. The SMoE gating mechanism is defined as:

$$G(x) := \text{Softmax}(\text{TopK}(x \cdot W_g)) \quad (4)$$

where $(\text{TopK}(l))_i := l_i$ if l_i is within the top-K values $l \in \mathbb{R}^n$ and $(\text{TopK}(l))_i := -\infty$ otherwise. This ensures that only the most relevant experts receive nonzero gating weights after applying the softmax function.

The final output of SMoE is then computed as:

$$\sum_{i=1}^n G(x)_i f_i(x) \quad (5)$$

Top-K experts selection in SMoE maintains model expressiveness while significantly improving efficiency.

3.2 Composition of Variant Experts via Hidden Factors (CoVE_h)

Here, we introduce the Composition of Variant Experts (CoVE), which differs from the traditional Mixture-of-Experts approaches in several key ways. First, instead of employing multiple instances of identical feed-forward network (FFN), CoVE leverages different types of experts. Particularly, these experts can include various recommendation models, such as collaborative filtering models (e.g., BPR [52]) for capturing user preferences and sequential models (e.g., GRU4Rec [22]) for modeling session-based interactions. Second, each expert interprets the same input data differently, offering diverse perspectives. For example, BPR treats input data as user-item interactions, while GRU4Rec models sequential behavior. Third, because each expert digests input data differently, during training, all experts are trained simultaneously to give all experts the ability to generalize; during inference, only the most relevant experts are activated. Fourth, we standardize the Compositions of Variant Experts so that each expert can determine its own input to the routing gate, which is then aggregated to produce the input to the gating mechanism. More details will be discussed in the next paragraphs.

Standardized Variant Experts. Various type of experts may produce various type of outputs from various type of inputs, providing unique insights into the data. For example, BPR learns from triplet user-positive-negative item interactions, optimizing rankings to favor positive items, providing robust user and item representations. In addition, GRU4Rec captures temporal user behaviors by predicting the next item in a sequence of interactions, relying solely on item sequence for input and optimizing the ranking of the next item accordingly. By integrating diverse expert models, we achieve a more comprehensive understanding of user behavior and preferences.

Despite their differences, we standardize the output of all experts including three components:

- Hidden context representation: d -dimensional vector that captures the user u context up to time T

$$h_i(u, T) = \Phi_i(u, C_{1:T}^u) \in \mathbb{R}^d \quad (6)$$

- Item embeddings: $\Psi(p) \in \mathbb{R}^d$ is a d -dimensional vector representation of item p
- Item bias: $\beta(p) \in \mathbb{R}$ is a scalar representing the bias of item p

Each expert serves as a sub-model. Using a gating mechanism, the three components from all experts are aggregated to form the final representations. We named this approach CoVE_h as the aggregated components are hidden representation from variant of experts.

Gating Mechanism. The gating mechanism determines the relevance of each expert based on the input data, enabling the model to dynamically select the most appropriate experts for a given context. More concretely, the contribution of each expert is decided upon the model receiving the inputs via the gating mechanism. By learning the contribution of each expert, the model can adapt to diverse user behaviors and preferences, ultimately enhancing recommendation quality.

Each of the n experts determines its own d -dimensional input to the gating mechanism, denoted as $e_i(u, C_{1:T}^u)$. These individual inputs are then aggregated to form an $n \times d$ -dimensional input for the gate. This design allows for flexibility in selecting the most relevant features for the gating mechanism, tailored to each expert. For instance, given a specific input, user representations might be crucial for the gate in BPR, whereas GRU4Rec may utilize embedded input items as gate inputs. Once computed, all experts' gate inputs are aggregated and passed through the gating mechanism.

In this work, we employ the simple gate mechanism consisting of a fully connected layer with a softmax activation function. This ensures that the gate assigns a weight to each expert while maintaining a total sum of 1. Particularly, the gating mechanism can be represented as:

$$g(u, C_{1:T}^u) := \text{Softmax} \left(W_g \left[e_i(u, C_{1:T}^u) \right]_1^n \right) \quad (7)$$

where $e_i(u, C_{1:T}^u)$ is the i -th expert's gate input, $\left[e_i(u, C_{1:T}^u) \right]_1^n$ denotes the concatenation of all gate inputs, and W_g is the trainable weights. The output of gate $g(u, C_{1:T}^u) \rightarrow \mathbb{R}^n$ is subsequently used to weigh the contributions of the experts' outputs to the final representation.

Training Variant Experts. Our CoVE framework enables each expert to process input data differently. During training, all experts are optimized simultaneously to collectively provide a comprehensive view of the data. This approach ensures that each expert learns distinct aspects of the data, contributing to the final recommendation.

The gating mechanism determines the contribution of each expert based on the input data. The final hidden representation, item embedding, and bias term are computed as weighted sums of the corresponding components from all experts, where the weights are determined by the gating function:

$$h(u) = \sum_{i=1}^n g(u, C_{1:T}^u)_i \times h_i(u, T) \quad (8)$$

$$\Psi(p) = \sum_{i=1}^n g(u, C_{1:T}^u)_i \times \Psi_i(p) \quad (9)$$

$$\beta(p) = \sum_{i=1}^n g(u, C_{1:T}^u)_i \times \beta_i(p) \quad (10)$$

where $g(u, C_{1:T}^u)$ is derived from Equation 7, and $h_i(u, T), \Psi_i(p), \beta_i(p)$ represent the hidden context, item embedding, and item bias deriving from the i -th expert.

The final item score for a user u and item p is computed by taking the dot product between the hidden representation and item embedding, and then adding the item bias:

$$\hat{r}_{u,p} = h(u) \cdot \Psi(p) + \beta(p) \quad (11)$$

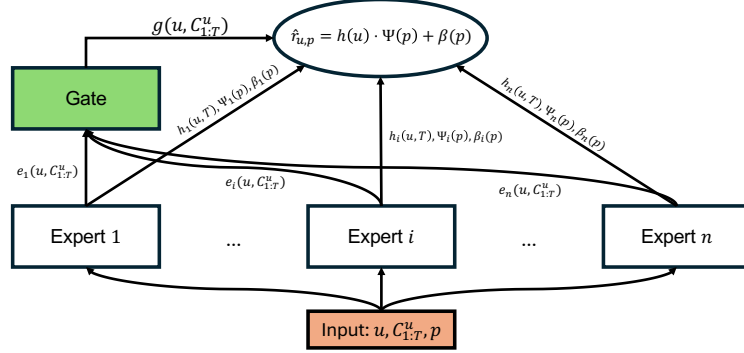


Fig. 4. Overall flow of CoVE_h variant. All experts receive user identity u with their historical interactions $C_{1:T}^u$ and candidate item p . Each expert i then returns the current context representation $h_i(u, T)$, item embeddings $\Psi_i(p)$, and item biases $\beta_i(p)$.

Learning objective. The general learning objective is to enhance ranking performance between positive and negative items in the candidate set. Its primary objective is to maximize the probability that positive items are ranked higher than negative ones. This can be achieved by minimizing the negative log-likelihood:

$$\ell = -\frac{1}{N_I} \log \sigma(\hat{r}_i - \hat{r}_j) \quad (12)$$

where $\hat{r}_i$ and $\hat{r}_j$ denote the predicted scores for a positive item (a preferred item) and a negative item (an unobserved or less preferred item), respectively, and N_I is the total number of item pairs. This formulation treats all negative items equally, ensuring uniform optimization across the training samples.

In practice, however, the learning dynamics differ between popular and long-tail items. Popular items are frequently sampled and updated during training, while long-tail items (those with fewer interactions) receive less attention. As a result, the prediction scores for long-tail items tend to remain low, making them “easy negatives” that do not significantly impact the optimization process. By contrast, distinguishing between positive items and popular negative items is more challenging.

To address this imbalance, BPR-max loss incorporates the softmax score corresponding to each negative item in the candidate set so that it assigns lower weights to these easy negatives, allowing the model to focus more on harder negatives, minimizing the following loss:

$$\ell = -\log \sum_i \sum_{j=1}^{N_I} s_j \sigma(\hat{r}_i - \hat{r}_j) \quad (13)$$

where $s_j = \frac{\exp(\hat{r}_j)}{\sum_{k=1}^{N_I} \exp(\hat{r}_k)}$ is the normalization score of the negative item j via the Softmax function. By prioritizing these challenging comparisons, the model promotes more effective learning and improves its ability to rank positive items accurately across a diverse set of candidates.

Inference. In our unique framework, each expert observes input data differently, so in the training phase, we employed the continuous gate mechanism. During inference, we derive the final prediction using sparse gate mechanism, where only the most relevant experts are activated based on the input data. This routing mechanism allows the model to dynamically select the most appropriate experts for the given context, reducing the computational cost since there only k experts were selected. We entrust the routing mechanism to choose the best experts. Following Equation 5, the hidden representation during inference is computed as:

$$h(u) = \sum_{i=1}^n G(u, C_{1:T}^u)_i f_i(u, C_{1:T}^u) \quad (14)$$

where $G(u, C_{1:T}^u)$ is the sparse gate defined in Equation 4.

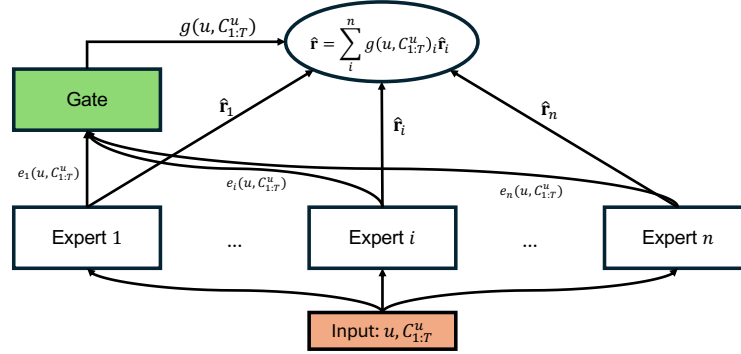


Fig. 5. Overall flow of CoVE_s variant. All experts receive user identity u with their historical interactions $C_{1:T}^u$ and candidate item p . Here, each expert i just returns its predicted scores for the set of candidate items $\hat{r}_i$.

3.3 Composition of Variant Experts via Scoring Function (CoVE_s)

We propose another approach for aggregation that offers greater flexibility but shallower expert connections. Unlike CoVE_h which experts are combined at *hidden* representation level, this approach, referred to as CoVE_s, gathers item *scores* predictions from individual experts.

Specifically, each expert generates predicted item scores directly, which are subsequently combined using a weighted sum via the same routing mechanism. This method offers greater flexibility compared to the CoVE_h approach, as it does not require experts to share the same hidden dimensions, provided that an expert is capable of providing scores over the item set. This approach bears resemblance to a learnable ensemble method, yet it possesses the advantage of trainable aggregation weights that can adapt dynamically to varied contexts. Consequently, the application of the same set of experts may result in varying weights, depending on the specific context.

The final ranking scores for all items are formally computed using the gating aggregation as follows:

$$\hat{r} = \sum_{i=1}^N g(u, C_{1:T}^u)_i \hat{r}_i \quad (15)$$

where $\hat{\mathbf{r}}_i \in \mathbb{R}^N$ denotes the scores of all items provided by the i -th expert.

4 Related Work

In this section, we first survey related work that deals with general personalized recommendation (learning long-term preferences) and sequential recommendation (learning short-term preferences). In addition, we also survey existing LLM approaches for recommendations. Lastly, we describe related mixture-of-experts.

Personalized Recommendation. The general recommender systems produce a list of suggested items for a target user. A crucial part is to model long-term user preferences for personalized recommendations. Many works are based on Matrix Factorization [24, 30, 42, 52] that leverage past user-item interactions. In addition, many others try to address the sparsity issue using auxiliary information such as social networks [39], topic modeling [62], reviews [59, 73], questions [33], images [8], graphs [17, 26], etc. Within the scope of this work, we evaluate our framework with user-item interaction data only. The basis of the recommendation optimization objective is to estimate missing ratings by minimizing the reconstruction of observed scores, e.g., MF [30], or infer ranking scores by maintaining the favoring of positive items over negative items, e.g., BPR [52]. In this work, we concentrate on the latter, which is more suitable for implicit feedback data (e.g., clicks, views, etc.).

Sequential Recommendation. Sequential recommendation (SR), which aims to predict a user’s next action based on their historical temporal interactions, has attracted significant attention in recent years. This is particularly fueled by the rise of deep learning techniques and the growing relevance of sequential patterns in modern user behaviors. Sequential recommendation has evolved into various taxonomies, including (i) pure ID-based SR, (ii) multi-modal SR, (iii) cross-domain SR, and (iv) LLM-powered SR. Of particular interest to this work is the pure ID-based SR, which focuses on modeling order and dependency of user-item interactions without additional information such as text, images, or other modalities. Prior works have explored various approaches to model sequential data, ranging from traditional methods such as Markov Chains [53, 65] and translation-based [16]; modeling repeating behavior [2, 50]; to deep learning, including recurrent neural networks (RNNs) [21, 22], Transformer-based architectures [13, 28, 56]. FPMC [53] subsumes matrix factorization with Markov Chains to model personalized next-basket recommendation. TransRec [16] assumes and learns a transition representation: item + user $\approx$ next item. Leveraging the success of RNNs in sequential modeling, GRU4Rec [22] uses gated recurrent unit (GRU) to model session data, capturing the temporal dynamics of user behavior. Based on the initial success, GRU4Rec+ [21] extends its predecessor by incorporating a novel negative sampling strategy and new ranking loss functions. Convolutional neural networks (CNNs) [6, 57, 68, 71] have also been applied to sequential recommendation to capture local patterns in user behavior, enabling faster parallel computation. Graph-based approaches [63, 64, 69] have been explored due to the flexibility in modeling complex social relationships and transition patterns using graph neural networks. With the advent of attention and Transformer-based architectures [60], sequential recommendation has seen significant advancements [28, 44, 56, 58, 61, 70]. NARM [34] and STAMP [37] utilize attention mechanisms to model the importance of different items *within the same session*, creating *long-term* or *local* representation, which is then combined with the last hidden factors, referred to as *short-term* or *global* feature. These long-term and short-term features differ from our definitions of short- and long-term preference models in section 2. Transformer models, such as SASRec [28] and BERT4Rec [56], have been widely adopted due to their ability to capture long-range dependencies and parallelize computations. Transformers4Rec [13] facilitates the use of Transformer-based architectures in sequential recommendation by providing a unified library for various Transformer-based models, using HuggingFace Frameworks [66]. To this date, Transformers-based models and their variants remain one of the most popular and effective approaches for sequential recommendation [1, 46–48].

Despite the popularity, recent studies on the reproducibility of recommender systems have shown that many models do not achieve their full potential due to issues with (i) implementation correctness [19, 41, 45], (ii) hyper-parameter tuning [55], or (iii) the use of different loss functions [21, 29]. [45] have done hypothesis testing and concluded that the “superior” BERT4Rec is not systematically better than SASRec in the published literature, and that the performance gap is mainly due to the poor replicability. [29] shows that the performance gap between SASRec and BERT4Rec is due to the use of different loss functions, where SASRec uses binary cross-entropy (BCE) loss and BERT4Rec is trained using cross-entropy (CE) loss. They further show that the performance of SASRec can be improved and outperform BERT4Rec by using CE loss. [13, 29, 45] also show that BPR [52] and GRU4Rec [21, 22] are simple yet strong baselines for sequential recommendation, and can achieve competitive results with BERT4Rec [56] with proper negative sampling and loss function.

Other taxonomies of sequential recommendation, including multi-modal and cross-domain, are not the focus of this study. We refer to [7, 9, 36, 43] for more comprehensive surveys.

LLM for Recommendations. The utilization of a large language model (LLM) in recommendation systems has garnered significant attention in recent research. Language model architectures have been extensively adopted in recommendation systems due to their inherent similarities in sequential characteristics, particularly in sequential recommendation tasks, as evidenced by models such as SASRec [28] and BERT4Rec [56]. The incorporation of large language models (LLMs) into recommendation systems is a growing trend. This integration is primarily driven by the LLMs’ capacity for textual understanding, which enables text-based recommendations and the incorporation of textual modalities into recommendation systems [14, 51]. However, their role in ID-based recommendation remains limited, often functioning as a supplementary component rather than a core model [5]. There is no clear evidence showing that LLM can easily outperform traditional models in ID-based recommendation [23, 35, 72]. On the other hand, the high computational cost and input token limitations for datasets with large number of items outweigh their performance benefits, making their integration a challenging trade-off.

Mixture-of-Experts models have recently garnered significant attention, particularly following the success of [27], with various applications in natural language processing. The existing Mixture-of-Experts for recommendation research usually tackles multi-task learning formulation [40, 67], where the shared bottom networks contain general information while experts are trained for specific tasks. The routing mechanism will then select suitable experts for the task of given input. Orthogonally, [3] proposes a mixture of three experts for multi-modal sequential recommendation, including text, ID, and image experts. Comparisons with these MoE models are beyond the scope of this study.

5 Experiments

In this section, we evaluate our proposed CoVE in the task of session-aware recommendation, mainly on two publicly available benchmark datasets. The improvement in ranking performance validates the effectiveness of our solution.

5.1 Experimental Setup

Datasets. Recent studies have identified several widespread flaws in recommendation systems, including dataset-task mismatch and negative sampling during testing, which hinder accurate evaluation [20, 31, 57]. In this paper, we strive to follow best practices to enhance the effectiveness of model evaluation.

Table 2. Data statistics

	Diginetica	RetailRocket
#interactions	12,146	230,817
#users	571	4,249
#sessions	2,670	24,732
#items	6,008	36,658
#sessions per user	4.68	5.82
#interactions per item	2.02	6.30
#interactions per session	4.55	9.33
density	0.354%	0.148%

To address the dataset-task mismatch, we utilize session-aware datasets³. In these datasets, each user interacts with multiple items within a browsing *session*, and may have several such sessions recorded. Specifically, our primary experiments are conducted on two datasets, Diginetica and RetailRocket, as detailed in [section 2](#).

Furthermore, during the evaluation phase, we score and rank items across the entire item set rather than relying on candidate sampling. This approach mitigates biases introduced by the selection of candidate sets [4, 12, 31], ensuring fairer comparisons and improved reproducibility.

For statistical sufficiency, we retain users with at least three sessions and items with at least five interactions. The last two sessions of each user are reserved for validation and testing, while the remaining sessions are used for training. These two reserved sessions are randomly assigned to validation and test sets. The dataset statistics are presented in Table 2. Code and data for reproducibility are publicly available⁴.

Evaluation metrics. To evaluate the effectiveness of each model, we let each model produce a ranked list of recommended items for every user. We quantitatively evaluate ranking performance using various well-known ranking metrics, including Area Under the ROC Curve (AUC), Mean Reciprocal Rank (MRR), Normalized Discounted Cumulative Gain (NDCG), and Recall. For these metrics, higher values indicate better quality. Comparisons between methods are tested with one-tailed paired-sample Student’s t-test at 0.05 level (i.e., p -value < 0.05).

Expert Selection. As mentioned in [section 4](#), sequential recommendation models may not achieve their full replicability and reproducibility due to flaws in implementation and hyper-parameter tuning [19, 41, 45, 55]; while simple yet effective models such as BPR [52] and GRU4Rec [21, 22] can achieve competitive results with proper training settings [13, 29, 45]. Furthermore, [32, 38] have demonstrated that shallow models can outperform deep models in various settings. In our experiments, we not only focus on deep models but also on popular and effective shallow models as experts, including two short-term preference models and two long-term preference models:

- Short-term preference models as expert:
 - **GRU4Rec** [21, 22]⁵: GRU4Rec is a session-based recommendation model that employs a Gated Recurrent Unit (GRU) to capture sequential user behavior. Benchmarks indicate that GRU4Rec outperforms more recent session-based recommendation models.
 - **SASRec++**: SASRec [28] is a session-based recommendation model that leverages self-attention mechanisms to capture user preferences. The SASRec+ [29] variant using Cross-Entropy loss can outperform SASRec with

³In contrast, previous studies converted user ratings into session data for evaluation. However, since these rating-based datasets were not inherently session-specific and may exhibit weaker sequential signals [57], they were excluded from this study.

⁴<https://github.com/PreferredAI/CoVE.git>

⁵In our experiment, we use the improved GRU4Rec model proposed in [21].

Binary Cross-Entropy loss and the allegedly superior BERT4Rec [56]. In this paper, we employ the BPR-max loss for SASRec, named SASRec++, which yields the best performance.

- Long-term preference models as expert:
 - **BPR** [52]: BPR is a collaborative filtering model that optimizes the ranking of positive items over negative ones. BPR is a simple yet effective model widely used in recommendation systems.
 - **FPMC** [53]: Factorized Personalized Markov Chains is a sequential recommendation model that captures user preferences by modeling item transitions within a session.

Comparative Methods. To sufficiently evaluate the effectiveness of our proposed method, we compare the proposed CoVE_h and CoVE_s models with individual selected short-term and long-term preference models as well as:

- **BERT4Rec** [56] is a session-based recommendation model that employs a Transformer architecture to capture sequential user behavior. BERT4Rec is included as a baseline due to its popularity and effectiveness in session-based recommendation.
- **LightGCN** [18] is a graph-based recommendation model that captures user-item interactions through a light-weight Graph Convolutional Network (GCN). LightGCN is included as a baseline due to its effectiveness in capturing user-item relationships. LightGCN will also be used as a long-term preference model in our investigation of CoVE_s with graph-based experts.
- **HGRU4Rec** [49] is a hierarchical GRU-based method for session-aware recommendation which take long-term user preferences into account. HGRU4Rec is included as a baseline due to its similarity in modeling both short- and long-term preferences.

Hyper-parameter Tuning. For all baselines, we perform a grid search with various hyper-parameters to find the best configuration. For CoVE_h and CoVE_s, we also conduct a grid search to determine the optimal configuration. We use the validation set to tune the hyper-parameters to their best and the test set to evaluate model performance. The main hyper-parameters include:

- **Loss Function.** We consider various loss functions, including BPR [52], BPR-max [21], binary cross-entropy loss [28], cross-entropy [56], top-1 [22]. Generally, BPR and BPR-max losses perform the best based on their ability to distinguish positive from negative samples.
- **Learning Rate.** We consider learning rates in the range of [0.005, 0.1].
- **Batch Size.** We consider batch sizes in the range of [32, 512]. Different loss functions may require different batch sizes. For example, BPR-max loss fits well with a smaller batch size of around [32, 128], while BPR-loss can speed up with a larger batch size of 256 or 512 without trade-off in performance.

Experts' Gate Input. As described in Section 3, we give the freedom of deciding gate input to the experts. Individual experts can use their own unique features as gate input by a standardized function. In the experiments for CoVE_h and CoVE_s, we follow common practice in NLP research [27], where the input to the gate is item embeddings.

5.2 Overall Performance

Here, we investigate whether using a mixture of experts enhances recommendation effectiveness compared to individual experts and baseline models.

Results reported in Table 3 consistently show that mixtures of experts outperform the baselines across datasets in terms of MRR, NDCG, and Recall significantly. The AUC metric does not fully reflect recommendation performance, measuring the proportion of pairwise ranking correctly. CoVE_h and CoVE_s maintain high AUC results, close to the

Table 3. Overall ranking performance: Comparison of CoVE against baselines across datasets

	Model	AUC	MRR	NDCG@k		Recall@k	
				k=10	k=20	k=10	k=20
Diginetica	GRU4REC	<u>0.7771</u>	<u>0.2979</u>	0.3171	0.3233	0.3888	0.4133
	SASREC	0.7666	0.2943	<u>0.3301</u>	<u>0.3343</u>	<u>0.4466</u>	<u>0.4641</u>
	BERT4Rec	0.7177	0.3426	0.3560	0.3575	0.4011	0.4063
	BPR	0.6334	0.0748	0.0854	0.0961	0.1361	0.1775
	FPMC	0.6609	0.0945	0.1107	0.1205	0.1805	0.2189
	LightGCN	0.7547	0.2120	0.2488	0.2601	0.3835	0.4273
	HGRU4REC	0.5606	0.1188	0.1333	0.1392	0.1893	0.2130
	CoVE _s	0.7854	0.4112 [§]	0.4379 [§]	0.4416 [§]	0.5254 [§]	0.5394 [§]
	CoVE _h	0.8007 [§]	0.3756 [§]	0.4003 [§]	0.4060 [§]	0.4869 [§]	0.5096 [§]
	Improvement %	3.04%	38.03%	32.66%	32.10%	17.64%	16.22%
RetailRocket	GRU4REC	<u>0.8772</u>	<u>0.2687</u>	<u>0.3035</u>	<u>0.3175</u>	<u>0.4356</u>	<u>0.4909</u>
	SASREC	0.6606	0.0962	0.1334	0.1412	0.2594	0.2892
	BERT4Rec	0.7827	0.2278	0.2551	0.2596	0.3476	0.3653
	BPR	0.8035	0.2161	0.2451	0.2528	0.3493	0.3794
	FPMC	0.8393	0.2437	0.2744	0.2848	0.3879	0.4290
	LightGCN	0.8044	0.1065	0.1238	0.1333	0.1970	0.2346
	HGRU4REC	0.8692	0.2327	0.2612	0.2731	0.3735	0.4203
	CoVE _s	0.8767	0.3365 [§]	0.3675 [§]	0.3783 [§]	0.4813 [§]	0.5239 [§]
	CoVE _h	0.8798 [§]	0.3406 [§]	0.3732 [§]	0.3831 [§]	0.4928 [§]	0.5319 [§]
	Improvement %	0.30%	26.76%	22.97%	20.66%	13.13%	8.35%

[§] p -value < 0.05. The statistical test is performed against the best-performing baseline. The best values are **bolded**. The best values among baselines are underlined.

best reported AUC yet still statistically significant. Similarly, CoVE_s also achieves the best performance across datasets. Among the selected experts, GRU4REC remains the best-performing model overall, showing its robustness in the sequential recommendation task. The session-aware baseline HGRU4REC does not outperform GRU4REC, which has also been reported in some benchmark [32].

FPMC itself can be interpreted as a combination of Matrix Factorization (MF) and Factorization Markov Chains (FMC), which is similar to a composition of one long-term (MF) and one short-term (FMC) expert. However, each component contributes equally towards final optimization objective.

5.3 Ablation Study

In this section, we systematically study the contribution of different components to our CoVE. First, we investigate the effectiveness of the proposed gating mechanism. Then, we analyze various compositions of experts to gain insight in selecting the optimal combination of variant experts.

Gating effectiveness. To further emphasize the contribution of the proposed gating mechanism, we evaluate the performance of both variants of CoVE without the learnable weights, the gate values are always uniform values, i.e., $g(x)_i = \frac{1}{K}$, where K is the number of experts, which assuming all experts contribute equally towards the overall optimization objective. Table 4 reports the ranking performance of CoVE_h and CoVE_s with and without the learnable

Table 4. Gating effectiveness evaluation: Ranking performance of both variants of CoVE with and without gating mechanism

Dataset	Model	Gate	AUC	MRR	NDCG@k		Recall@k	
					k=10	k=20	k=10	k=20
Diginetica	CoVE _s	✓	0.7854	0.4112 [§]	0.4379 [§]	0.4416 [§]	0.5254 [§]	0.5394 [§]
		–	0.7694	0.3860	0.4071	0.4119	0.4816	0.5009
	CoVE _h	✓	0.8007	0.3756	0.4003	0.4060	0.4869 [§]	0.5096
		–	0.8022	0.3694	0.3939	0.3997	0.4799	0.5026
RetailRocket	CoVE _s	✓	0.8767 [§]	0.3365 [§]	0.3675 [§]	0.3783 [§]	0.4813	0.5239
		–	0.8726	0.2847	0.3279	0.3395	0.4794	0.5251
	CoVE _h	✓	0.8826	0.3419 [§]	0.3735 [§]	0.3851 [§]	0.4902 [§]	0.5357
		–	0.8836	0.3216	0.3549	0.3670	0.4782	0.5265

[§] p -value < 0.05. The best values are **bolded**.

weights from the proposed gating mechanism. Without the flexibility of controlling how much each expert contributes, we observed a degradation in the recommendation performance for both CoVE_s and CoVE_h in term of ranking performance. CoVE_h without the learnable gating achieves a bit higher AUC measure in both datasets, however, it is not statistically significant different from CoVE_h with learnable gating. Thus, balancing the contribution among variant experts is not a trivial task.

Same-Type Experts vs. Varying-Type Experts. The results from [section 2](#) indicate the presence of both short-term and long-term preferences within the session-aware recommendation context. We categorize the compositions of experts into two sub-categories: same-type experts and varying-type experts. The same-type expert mixture includes only short-term (or long-term) experts, while the varying-type expert mixture comprises both short-term and long-term experts. We further explore different compositions of variant experts, including combinations of 2, 3, and 4 experts with varying types (which include both short-term and long-term experts). The results for these compositions on the Diginetica dataset are presented in [Table 5](#), while compositions for the RetailRocket dataset are shown in [Table 6](#).

As demonstrated in [Table 5](#), compositions of the same-type experts for CoVE_h result in a performance degradation compared to individual experts, whereas compositions of varying types lead to significant improvements. For CoVE_s, it appears that it is easier to achieve better results compared to individual experts. The key factor is identifying the optimal expert composition; in this case, the selection of the optimal composition depends on the respecting metric.

The results in [Table 6](#) show a similar pattern, where the best compositions consistently include cross-type experts (GRU4REC+BPR) for both CoVE_h and CoVE_s.

5.4 CoVE_h With Graph-Based Expert

We further investigate the impact of incorporating a graph-based expert, LightGCN, as a long-term preference model within the CoVE_h framework. [Table 7](#) summarizes the experimental results on both Diginetica and RetailRocket datasets, comparing individual experts as well as CoVE_h based on BPR model.

On the Diginetica dataset, integrating LightGCN with short-term experts leads to notable improvements. Specifically, the combination of GRU4Rec and LightGCN achieves the highest AUC score (0.8139), outperforming all other configurations. Additionally, the pairing of SASRec with LightGCN achieves the best Recall@10 (0.4711) and Recall@20

Table 5. Ranking performance with varying different experts on Diginetica dataset. GRU and SAS are GRU4Rec and SASRec++ respectively.

	Number of Experts	Expert model				Ranking metrics					
		Short-term		Long-term		AUC	MRR	NDCG@k		Recall@k	
		GRU	SAS	BPR	FPMC			k=10	k=20	k=10	k=20
CoVE _h	2	✓	✓			0.7287	0.1569	0.1785	0.1876	0.2627	0.2977
				✓	✓	0.7350	0.2368	0.2730	0.2781	0.3958	0.4168
		✓		✓		0.7940	0.3507	0.3720	0.3803	0.4518	0.4851
			✓	✓		0.7921	0.2380	0.2727	0.2811	0.3940	0.4273
		✓			✓	0.8007	0.3756	0.4003	0.4060	0.4869	0.5096
			✓		✓	0.6928	0.1321	0.1504	0.1562	0.2224	0.2452
	3	✓	✓		✓	0.7419	0.2189	0.2341	0.2444	0.2977	0.3380
		✓	✓	✓		0.7800	0.2871	0.3138	0.3191	0.4063	0.4273
			✓	✓	✓	0.7769	0.2377	0.2708	0.2785	0.3870	0.4168
		✓		✓	✓	0.7806	0.3355	0.3620	0.3675	0.4536	0.4746
	4	✓	✓	✓	✓	0.7744	0.3291	0.3541	0.3591	0.4413	0.4606
CoVE _s	2	✓	✓			0.7930	0.4036	0.4318	0.4359	0.5236	0.5394
				✓	✓	0.7423	0.2502	0.2844	0.2893	0.4046	0.4238
		✓		✓		0.8119	0.3674	0.3845	0.3968	0.4553	0.5044
			✓	✓		0.8059	0.3652	0.3998	0.4082	0.5166	0.5499
		✓			✓	0.8006	0.3521	0.3726	0.3801	0.4501	0.4799
			✓		✓	0.7943	0.3720	0.4045	0.4145	0.5166	0.5569
	3	✓	✓		✓	0.7860	0.4098	0.4362	0.4401	0.5219	0.5377
		✓	✓	✓		0.7833	0.4097	0.4385	0.4428	0.5324	0.5499
			✓	✓	✓	0.7919	0.3578	0.3980	0.4023	0.5289	0.5464
		✓		✓	✓	0.8035	0.3720	0.3897	0.4000	0.4606	0.5009
	4	✓	✓	✓	✓	0.7854	0.4112	0.4379	0.4416	0.5254	0.5394

The best values are **bolded**.

(0.4956). These results demonstrate that LightGCN as the graph-based long-term expert can complement short-term models and enhance overall recommendation.

In contrast, on the RetailRocket dataset, LightGCN as an individual expert does not perform as well as BPR. Consequently, combinations involving LightGCN do not outperform those with BPR in most metrics. However, the combination of SASREC and LightGCN still achieves competitive results, ranking second-best in several Recall and NDCG metrics, despite SASREC and LightGCN performing poorly individually. This suggests that even when individual experts are weak, their combination can provide complementary benefits.

Overall, these findings highlight the potential of integrating graph-based long-term experts, such as LightGCN, within the CoVE_h framework, particularly when paired with strong short-term models. This integration can lead to improved recommendation performance, similar to the results achieved with long-term preference models like BPR.

Table 6. Ranking performance with varying experts on RetailRocket dataset.

	Number of Experts	Expert model				Ranking metrics					
		Short-term		Long-term		AUC	MRR	NDCG@k		Recall@k	
		GRU	SAS	BPR	FPMC			k=10	k=20	k=10	k=20
CoVE _h	2	✓	✓			0.8782	0.3397	0.3712	0.3815	0.4869	0.5277
				✓	✓	0.7782	0.2221	0.2515	0.2595	0.3573	0.3886
		✓		✓		0.8798	0.3406	0.3732	0.3831	0.4928	0.5319
			✓	✓		0.8373	0.2224	0.2484	0.2593	0.3474	0.3904
		✓			✓	0.8853	0.3393	0.3716	0.3823	0.4898	0.5321
			✓		✓	0.8212	0.1791	0.1985	0.2082	0.2777	0.3163
	3	✓	✓		✓	0.8467	0.2942	0.3218	0.3316	0.4255	0.4639
		✓	✓	✓		0.8822	0.3337	0.3662	0.3769	0.4853	0.5277
			✓	✓	✓	0.7842	0.1600	0.1800	0.1880	0.2570	0.2885
		✓		✓	✓	0.8680	0.3207	0.3537	0.3644	0.4740	0.5159
	4	✓	✓	✓	✓	0.8644	0.3126	0.3439	0.3545	0.4582	0.5001
CoVE _s	2	✓	✓			0.8767	0.3365	0.3675	0.3783	0.4813	0.5239
				✓	✓	0.8150	0.2023	0.2338	0.2440	0.3490	0.3888
		✓		✓		0.8898	0.3354	0.3694	0.3814	0.4952	0.5425
			✓	✓		0.8283	0.1820	0.2404	0.2524	0.4354	0.4825
		✓			✓	0.8877	0.3087	0.3451	0.3577	0.4792	0.5288
			✓		✓	0.8269	0.1489	0.2036	0.2179	0.3921	0.4481
	3	✓	✓		✓	0.8038	0.2593	0.2824	0.2949	0.3742	0.4239
		✓	✓	✓		0.8235	0.2731	0.3070	0.3185	0.4297	0.4749
			✓	✓	✓	0.8129	0.1560	0.2129	0.2261	0.4048	0.4566
		✓		✓	✓	0.8828	0.3199	0.3546	0.3663	0.4815	0.5274
	4	✓	✓	✓	✓	0.8840	0.2379	0.2931	0.3067	0.4841	0.5382

The best values are **bolded**.

5.5 Case Study

We investigate whether the same items can be recommended based on variant experts. For example, an item aligned with one user’s long-term interests might simultaneously be recommended due to another user’s short-term preferences.

In the Diginetica dataset (see Figure 6), item 1357 serves as a positive example, correctly recommended to three users: indices 25, 272, and 273. Interestingly, the underlying contributions for recommending item 1357 vary across these users. For user 25, the gate value contributions are (0.0138, 0.2435, 0.0262, 0.7166) for GRU4Rec, SASRec, BPR, and FPMC, respectively. For user 272, the gate values are (0.0163, 0.5362, 0.0264, 0.4212). For user 273, they are (0.0258, 0.5925, 0.0405, 0.3411).

For users 272 and 273, SASRec emerges as the dominant expert recommending item 1357, with slightly different weightings, suggesting that item 1357 aligns with their short-term preferences. In contrast, for user 25, FPMC is the primary contributor, indicating that item 1357 likely reflects this user’s long-term preference.

Table 7. CoVE_h experimental results with graph-based expert

	Number of Experts	Expert model				Ranking metrics					
		Short-term		Long-term		AUC	MRR	NDCG@k		Recall@k	
		GRU	SAS	BPR	LightGCN			k=10	k=20	k=10	k=20
Diginetica	1	✓				0.7771	0.2979	0.3171	0.3233	0.3888	0.4133
			✓			0.7666	0.2943	0.3301	0.3343	0.4466	0.4641
				✓		0.6334	0.0748	0.0854	0.0961	0.1361	0.1775
					✓	0.7547	0.2120	0.2488	0.2601	0.3835	0.4273
	2	✓		✓		<u>0.7940</u>	0.3507	0.3720	0.3803	<u>0.4518</u>	<u>0.4851</u>
			✓	✓		0.7921	0.2380	0.2727	0.2811	0.3940	0.4273
RetailRocket	1	✓				<u>0.8772</u>	0.2687	0.3035	0.3175	0.4356	<u>0.4909</u>
			✓			0.6606	0.0962	0.1334	0.1412	0.2594	0.2892
				✓		0.8035	0.2161	0.2451	0.2528	0.3493	0.3794
					✓	0.8044	0.1065	0.1238	0.1333	0.1970	0.2346
	2	✓		✓		0.8798	0.3406	0.3732	0.3831	0.4928	0.5319
			✓	✓		0.8373	0.2224	0.2484	0.2593	0.3474	0.3904
		✓			✓	0.8344	0.2386	0.2640	0.2752	0.3634	0.4079
			✓		✓	0.8627	<u>0.3027</u>	<u>0.3318</u>	<u>0.3432</u>	<u>0.4425</u>	0.4872

The best values are **bolded** and second best values are underlined.

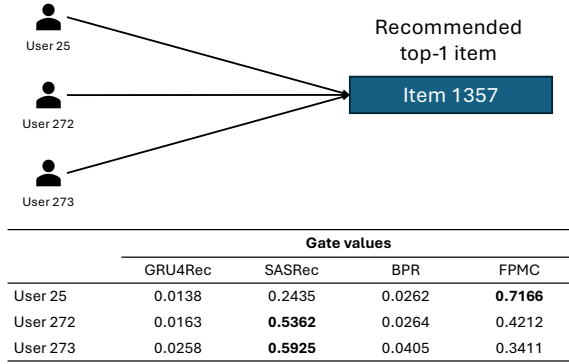


Fig. 6. An example of different gate values for different users being recommended the same item

5.6 LLM as Gating Mechanism

In this section, we explore the potential of using Large Language Models (LLMs) as a gating mechanism for CoVE. The objective is to study if LLMs can dynamically select experts based on user and item information, potentially improving recommendation performance.



Fig. 7. ChatGPT-4o as a gating mechanism for CoVE.

We ask ChatGPT to give a list of top-50 items based on the scores from the pretrained experts, all user historical interactions⁶. Figure 7 shows the prompt we used to query ChatGPT-4o. The LLM is expected to analyze the scores from each expert and the user's historical interactions to make its recommendations.

Response from ChatGPT-4o is shown in Figure 7b and Figure 7c, where it computes the average score assigned by each expert and ranks the items accordingly. The top-50 items are then presented as a recommendation list. This corresponds to the ablation study of gating mechanism discussed in subsection 5.3. The results suggest that, at present, ChatGPT does not provide additional benefits in weighing the experts when acting independently. This leaves opportunities for further exploration in future research.

5.7 Discussion

In this section, we further discuss alternative ways to scale up our proposed CoVE using a larger model (such as increasing the number of experts) or a larger dataset. In addition, we enumerate a few limitations and potential direction for future improvements.

Increasing the number of experts by multiplying the same set of experts. Here we try another approach to increase the number of experts in CoVE, by multiplying the same set of experts multiple times. As reported in Table 8, it is not clear that increasing the number of the same experts multiple times helps in enhancing recommendation performance. However, this approach also increases the computational cost of training due to the larger number of experts being used.

Scale to larger dataset. Here we would like to assess whether the overall performance of the proposed CoVE is still maintained on a larger scale of data. For this experiment, we use Cosmetics dataset⁷, containing more than 2.5 millions interactions (approximately 200 times bigger than Diginetica and 10 times bigger than RetailRocket dataset) with a total of 17,268 users, 42,367 items, and 172,242 sessions. Due to the larger number of users and items, the model size of each individual expert increases, taking much more time to train, especially with the continuous gating. We can get

⁶<https://chatgpt.com/share/68511ca8-605c-8012-ac49-2a4aba7f8f66>

⁷<https://www.kaggle.com/mkechinov/e-commerce-events-history-in-cosmetics-shop>

Table 8. Performance when increasing the number of experts: AUC and MRR

Multiply	Diginetica				RetailRocket			
	CoVE _h		CoVE _s		CoVE _h		CoVE _s	
	AUC	MRR	AUC	MRR	AUC	MRR	AUC	MRR
×1	0.8007	0.3756	0.7854	0.4112	0.8798	0.3406	0.8767	0.3365
×2	0.7982	0.3731	0.7964	0.4097	0.8826	0.3419	0.8755	0.3374
×4	0.7984	0.3842	0.7963	0.4125	0.8797	0.3406	0.8738	0.3383

Table 9. Performance on Cosmetics dataset with > 2.5 millions interactions

Model	AUC	MRR	NDCG@k		Recall@k	
			k=10	k=20	k=10	k=20
BPR	0.8054	0.0175	0.0189	0.0250	0.0396	0.0638
FPMC	0.9248	0.0985	0.1152	0.1319	0.2022	0.2684
GRU4REC	0.9147	<u>0.1568</u>	<u>0.1842</u>	<u>0.2042</u>	<u>0.3069</u>	<u>0.3857</u>
SASREC	0.9104	<u>0.1568</u>	0.1823	0.1999	0.2967	0.3661
HGRU4REC	0.8948	0.0764	0.0878	0.1020	0.1546	0.2113
CoVE _s	0.9167 [§]	0.1643 [§]	0.1937 [§]	0.2135 [§]	0.3224 [§]	0.4007 [§]
CoVE _h	0.9182 [§]	0.1647[§]	0.1940[§]	0.2139[§]	0.3229[§]	0.4016[§]
Improvement %	-0.71%	5.04%	5.32%	4.75%	5.21%	4.12%

[§]*p*-value < 0.05. The best values are **bolded**. The best values among baselines are underlined. The statistical test is performed against the best-performing baseline.

rid of this problem as we perform offline training. During inference, we use top k experts, which is a more efficient approach. Specifically, as reported in Table 9, the best performance model of CoVE is achieved using top-1 expert in inference. Overall, Both CoVE variants achieve significant better performance over the selected baselines in term of MRR, NDCG@k, and Recall@k. Although FPMC performs the best in term of AUC, its performance on MRR, NDCG@k, and Recall@k metrics are significantly worse than CoVE. This may be due to the main pairwise ranking objective, which is technically to optimize AUC.

CoVE_h and CoVE_s choices. The two proposed CoVE frameworks offer different levels of combining experts. CoVE_h combines experts at the hidden layer, allowing for more complex interactions between expert outputs. This also limits the particular experts that can be used, as they must have the same output hidden dimension. In contrast, CoVE_s combines experts at the output layer, which is simpler and more interpretable. We can utilize any experts with different output hidden dimensions, as long as they can output a score for the set of items. The gating mechanism in CoVE_s is also simpler, as it only needs to combine the output scores of the experts rather than their hidden states. However, this simplicity comes at the cost of potentially losing some complex interactions between expert outputs that CoVE_h can capture. The choice between these two variants depends on the specific use case and the desired complexity of the model. For example, CoVE_h may be more suitable for tasks requiring complex interactions between expert outputs, while CoVE_s may be preferred for tasks where interpretability is crucial.

Table 10. Training time for Diginetica dataset, same batch size (32) and number of epochs (200) for all models.

Model	BPR	FPMC	GRU4REC	SASREC	Expert total	CoVE _h	CoVE _s
Training time (s)	449.256	1146.133	676.165	705.373	2976.927	4677.913	3770.832

Running time analysis. CoVE models are mixtures of all individual experts, so the overall training time is approximately the sum of all experts’ training time, overhead from the gating mechanism, and the time saved by requiring only a single backward pass for the entire model instead of separate backward passes for individual experts.

Table 10 reports the training time for the two CoVE variants that incorporate all four experts (BPR, FPMC, GRU4REC, and SASREC), along with the training time for each expert on Diginetica dataset. All models were trained for 200 epochs with a batch size of 32, and running time was measured in seconds. Among the individual experts, BPR required the least time to train (7.5 minutes), while FPMC required the most (19.1 minutes). GRU4REC and SASREC took similar training times (11.3 and 11.8 minutes, respectively). The total training time for all four experts was 49.6 minutes. In comparison, CoVE_h took 78 minutes, and CoVE_s took 62.8 minutes, corresponding to approximately 1.6 and 1.3 times the combined training time of the individual experts, respectively. This increase is expected, as CoVE models incur additional computational costs from the gating mechanism and the aggregation of expert outputs.

In practice, the use of pretrained individual experts can substantially reduce the training time for CoVE variants. When pretrained experts are available, they can be directly incorporated into CoVE without training from scratch, enabling much faster convergence of the CoVE models.

Limitations and potential improvements. Although both of CoVE variants achieve consistently better performance, some limitations persist in the chosen approaches. Firstly, CoVE uses continuous mixture-of-experts training, where every single expert is activated. This may result in a longer and inefficient training process and difficulties in scalability. However, due to the uniqueness of how each expert views input data, continuous training seems necessary for the experts to capture complete user behaviors. Secondly, various experts may converge differently, which the proposed methods did not tackle and could lead to overfitting for individual experts when the overall objective is achieved. The Composition of Variant Experts framework is generalized and flexible in incorporating any type of experts; however, in this work, expert selection mainly focuses on a few well-known approaches. Furthermore, we presuppose the output hidden dimension of each expert in the proposed CoVE_h variant to be the same. This limits the flexibility of each individual expert, as the best-performing parameters for one expert may differ from another in practice. Methods of combining experts with different hidden output dimensions could be considered.

6 Conclusion

While users typically have a long-term preference profile built up over a long period of time, there are short-term effects that may affect preferences as well. In this work, we establish that different types of ‘experts’ that focus on either short- or long-term preference would work better together, as not only do both types of preferences matter, but the way they do so differs from user to user. Which model predominates also varies across datasets, emphasizing the proposed framework approach of CoVE, leveraging on multiple experts working in concert, while accommodating a gating mechanism that modulates the influence of each expert on recommendations.

References

- [1] Davide Abbattista, Vito Walter Anelli, Tommaso Di Noia, Craig Macdonald, and Aleksandr Vladimirovich Petrov. 2024. Enhancing Sequential Music Recommendation with Personalized Popularity Awareness. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 1168–1173.
- [2] Rahul Bhagat, Srevatsan Muralidharan, Alex Lobzhanidze, and Shankar Vishwanath. 2018. Buy it again: Modeling repeat purchase recommendations. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 62–70.
- [3] Shuqing Bian, Xingyu Pan, Wayne Xin Zhao, Jinpeng Wang, Chuyuan Wang, and Ji-Rong Wen. 2023. Multi-modal Mixture of Experts Representation Learning for Sequential Recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (Birmingham, United Kingdom) (CIKM '23)*. Association for Computing Machinery, New York, NY, USA, 110–119. doi:10.1145/3583780.3614978
- [4] Rocío Cañamares and Pablo Castells. 2020. On target item sampling in offline recommender system evaluation. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 259–268.
- [5] Lei Chen, Chen Gao, Xiaoyi Du, Hengliang Luo, Depeng Jin, Yong Li, and Meng Wang. 2024. Enhancing ID-based Recommendation with Large Language Models. *ACM Trans. Inf. Syst.* (Nov. 2024). doi:10.1145/3704263 Just Accepted.
- [6] Qi Chen, Guohui Li, Quan Zhou, Si Shi, and Deqing Zou. 2022. Double attention convolutional neural network for sequential recommendation. *ACM Transactions on the Web* 16, 4 (2022), 1–23.
- [7] Shu Chen, Zitao Xu, WeiKe Pan, Qiang Yang, and Zhong Ming. 2024. A survey on cross-domain sequential recommendation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 7989–7998.
- [8] Xu Chen, Hanxiong Chen, Hongteng Xu, Yongfeng Zhang, Yixin Cao, Zheng Qin, and Hongyuan Zha. 2019. Personalized Fashion Recommendation with Visual Explanations based on Multimodal Attention Network: Towards Visually Explainable Recommendation. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (Paris, France) (SIGIR'19)*. Association for Computing Machinery, New York, NY, USA, 765–774. doi:10.1145/3331184.3331254
- [9] Xiaoqing Chen, Zitao Li, WeiKe Pan, and Zhong Ming. 2023. A survey on multi-behavior sequential recommendation. *arXiv preprint arXiv:2308.15701* (2023).
- [10] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555* (2014).
- [11] Aidan Clark, Diego de Las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, et al. 2022. Unified scaling laws for routed language models. In *International conference on machine learning*. PMLR, 4057–4086.
- [12] Alexander Dallmann, Daniel Zoller, and Andreas Hotho. 2021. A case study on sampling strategies for evaluating neural sequential item recommendation models. In *Proceedings of the 15th ACM Conference on Recommender Systems*. 505–514.
- [13] Gabriel de Souza Pereira Moreira, Sara Rabhi, Jeong Min Lee, Ronay Ak, and Even Oldridge. 2021. Transformers4rec: Bridging the gap between nlp and sequential/session-based recommendation. In *Proceedings of the 15th ACM conference on recommender systems*. 143–153.
- [14] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on recommender systems*. 299–315.
- [15] Hussein Hazimeh, Zhe Zhao, Aakanksha Chowdhery, Maheswaran Sathiamoorthy, Yihua Chen, Rahul Mazumder, Lichan Hong, and Ed Chi. 2021. Dselect-k: Differentiable selection in the mixture of experts with applications to multi-task learning. *Advances in Neural Information Processing Systems* 34 (2021), 29335–29347.
- [16] Ruining He, Wang-Cheng Kang, and Julian McAuley. 2017. Translation-based Recommendation. In *Proceedings of the Eleventh ACM Conference on Recommender Systems (Como, Italy) (RecSys '17)*. Association for Computing Machinery, New York, NY, USA, 161–169. doi:10.1145/3109859.3109882
- [17] Xiangnan He, Tao Chen, Min-Yen Kan, and Xiao Chen. 2015. TriRank: Review-aware Explainable Recommendation by Modeling Aspects. In *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management (Melbourne, Australia) (CIKM '15)*. Association for Computing Machinery, New York, NY, USA, 1661–1670. doi:10.1145/2806416.2806504
- [18] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [19] Balázs Hidasi and Ádám Tibor Czapp. 2023. The effect of third party implementations on reproducibility. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 272–282.
- [20] Balázs Hidasi and Ádám Tibor Czapp. 2023. Widespread flaws in offline evaluation of recommender systems. In *Proceedings of the 17th acm conference on recommender systems*. 848–855.
- [21] Balázs Hidasi and Alexandros Karatzoglou. 2018. Recurrent neural networks with top-k gains for session-based recommendations. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 843–852.
- [22] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. <http://arxiv.org/abs/1511.06939>
- [23] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning vector-quantized item representation for transferable sequential recommenders. In *Proceedings of the ACM Web Conference 2023*. 1162–1171.

- [24] Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE international conference on data mining*. Ieee, 263–272.
- [25] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation* 3, 1 (1991), 79–87.
- [26] Theis E. Jendal, Trung-Hoang Le, Hady W. Lauw, Matteo Lissandrini, Peter Dolog, and Katja Hose. 2024. Hypergraphs with Attention on Reviews for Explainable Recommendation. In *Advances in Information Retrieval*, Nazli Goharian, Nicola Tonello, Yulan He, Aldo Lipani, Graham McDonald, Craig Macdonald, and Iadh Ounis (Eds.). Springer Nature Switzerland, Cham, 230–246.
- [27] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [28] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [29] Anton Klenitskiy and Alexey Vasilev. 2023. Turning Dross Into Gold Loss: is BERT4Rec really better than SASRec?. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1120–1125.
- [30] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [31] Walid Krichene and Steffen Rendle. 2020. On sampled metrics for item recommendation. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 1748–1757.
- [32] Sara Latifi, Noemi Mauro, and Dietmar Jannach. 2021. Session-aware recommendation: A surprising quest for the state-of-the-art. *Information Sciences* 573 (2021), 291–315.
- [33] Trung-Hoang Le and Hady W. Lauw. 2024. Question-Attentive Review-Level Explanation for Neural Rating Regression. *ACM Trans. Intell. Syst. Technol.* 15, 6, Article 132 (Dec. 2024), 25 pages. doi:10.1145/3699516
- [34] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural attentive session-based recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1419–1428.
- [35] Ruyi Li, Wenhao Deng, Yu Cheng, Zheng Yuan, Jiaqi Zhang, and Fajie Yuan. 2023. Exploring the upper limits of text-based collaborative filtering using large language models: Discoveries and insights. *arXiv preprint arXiv:2305.11700* (2023).
- [36] Zihao Li, Chao Yang, Yakun Chen, Xianzhi Wang, Hongxu Chen, Guandong Xu, Lina Yao, and Michael Sheng. 2024. Graph and sequential neural networks in session-based recommendation: A survey. *Comput. Surveys* 57, 2 (2024), 1–37.
- [37] Qiao Liu, Yifu Zeng, Refuoe Mokhosi, and Haibin Zhang. 2018. STAMP: short-term attention/memory priority model for session-based recommendation. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1831–1839.
- [38] Malte Ludewig, Noemi Mauro, Sara Latifi, and Dietmar Jannach. 2019. Performance comparison of neural and non-neural approaches to session-based recommendation. In *Proceedings of the 13th ACM conference on recommender systems*. 462–466.
- [39] Hao Ma, Haixuan Yang, Michael R. Lyu, and Irwin King. 2008. SoRec: social recommendation using probabilistic matrix factorization. In *Proceedings of the 17th ACM Conference on Information and Knowledge Management (Napa Valley, California, USA) (CIKM '08)*. Association for Computing Machinery, New York, NY, USA, 931–940. doi:10.1145/1458082.1458205
- [40] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1930–1939.
- [41] Aleksandr Milogradskii, Oleg Lashinin, Alexander P, Marina Ananyeva, and Sergey Kolesnikov. 2024. Revisiting BPR: A Replicability Study of a Common Recommender System Baseline. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 267–277.
- [42] Andriy Mnih and Russ R Salakhutdinov. 2007. Probabilistic Matrix Factorization. In *Advances in Neural Information Processing Systems*, J. Platt, D. Koller, Y. Singer, and S. Roweis (Eds.), Vol. 20. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2007/file/d7322ed717dedf1eb4e6e52a37ea7bcd-Paper.pdf
- [43] Mahreen Nasir and Christie I Ezeife. 2023. A survey and taxonomy of sequential recommender systems for e-commerce product recommendation. *SN Computer Science* 4, 6 (2023), 708.
- [44] Aleksandr Petrov and Craig Macdonald. 2022. Effective and Efficient Training for Sequential Recommendation using Recency Sampling. In *Sixteen ACM Conference on Recommender Systems*.
- [45] Aleksandr Petrov and Craig Macdonald. 2022. A systematic review and replicability study of bert4rec for sequential recommendation. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 436–447.
- [46] Aleksandr Vladimirovich Petrov and Craig Macdonald. 2023. gsaRec: Reducing overconfidence in sequential recommendation trained with negative sampling. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 116–128.
- [47] Aleksandr Vladimirovich Petrov and Craig Macdonald. 2025. Improving Effectiveness by Reducing Overconfidence in Large Catalogue Sequential Recommendation with gBCE loss. *ACM Transactions on Recommender Systems* 3, 4 (2025), 1–34.
- [48] Aleksandr V Petrov, Craig Macdonald, and Nicola Tonello. 2025. Efficient Recommendation with Millions of Items by Dynamic Pruning of Sub-Item Embeddings. *arXiv preprint arXiv:2505.00560* (2025).
- [49] Massimo Quadrana, Alexandros Karatzoglou, Balázs Hidasi, and Paolo Cremonesi. 2017. Personalizing session-based recommendations with hierarchical recurrent neural networks. In *proceedings of the Eleventh ACM Conference on Recommender Systems*. 130–137.
- [50] Pengjie Ren, Zhumin Chen, Jing Li, Zhaochun Ren, Jun Ma, and Maarten De Rijke. 2019. Repeatnet: A repeat aware neural recommendation machine for session-based recommendation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 4806–4813.

- [51] Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Representation learning with large language models for recommendation. In *Proceedings of the ACM Web Conference 2024*. 3464–3475.
- [52] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence* (Montreal, Quebec, Canada) (UAI '09). AUAI Press, Arlington, Virginia, USA, 452–461.
- [53] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized markov chains for next-basket recommendation. In *Proceedings of the 19th international conference on World wide web*. 811–820.
- [54] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations*.
- [55] Faisal Shehzad and Dietmar Jannach. 2023. Everyone's a winner! on hyperparameter tuning of recommendation models. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 652–657.
- [56] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [57] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.
- [58] Md Mehrab Tanjim, Congzhe Su, Ethan Benjamin, Diane Hu, Liangjie Hong, and Julian McAuley. 2020. Attentive Sequential Models of Latent Intent for Next Item Recommendation. In *Proceedings of The Web Conference 2020* (Taipei, Taiwan) (WWW '20). Association for Computing Machinery, New York, NY, USA, 2528–2534.
- [59] Yi Tay, Anh Tuan Luu, and Siu Cheung Hui. 2018. Multi-pointer co-attention networks for recommendation. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2309–2318.
- [60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [61] Chenyang Wang, Weizhi Ma, Chong Chen, Min Zhang, Yiqun Liu, and Shaoping Ma. 2023. Sequential Recommendation with Multiple Contrast Signals. *ACM Trans. Inf. Syst.* 41, 1, Article 11 (Jan. 2023), 27 pages. doi:10.1145/3522673
- [62] Hao Wang, Binyi Chen, and Wu-Jun Li. 2013. Collaborative topic regression with social regularization for tag recommendation.. In *IJCAI*, Vol. 13. 2719–2725.
- [63] Jianling Wang, Kaize Ding, Liangjie Hong, Huan Liu, and James Caverlee. 2020. Next-item Recommendation with Sequential Hypergraphs. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval* (Virtual Event, China) (SIGIR '20). Association for Computing Machinery, New York, NY, USA, 1101–1110. doi:10.1145/3397271.3401133
- [64] Jianling Wang, Kaize Ding, Ziwei Zhu, and James Caverlee. 2021. Session-based recommendation with hypergraph attention networks. In *Proceedings of the 2021 SIAM international conference on data mining (SDM)*. SIAM, 82–90.
- [65] Pengfei Wang, Jiafeng Guo, Yanyan Lan, Jun Xu, Shengxian Wan, and Xueqi Cheng. 2015. Learning Hierarchical Representation Model for NextBasket Recommendation. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Santiago, Chile) (SIGIR '15). Association for Computing Machinery, New York, NY, USA, 403–412. doi:10.1145/2766462.2767694
- [66] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [67] Jiahui Xu, Lu Sun, and Dengji Zhao. 2024. MoME: Mixture-of-Masked-Experts for Efficient Multi-Task Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) (SIGIR '24). Association for Computing Machinery, New York, NY, USA, 2527–2531. doi:10.1145/3626772.3657922
- [68] An Yan, Shuo Cheng, Wang-Cheng Kang, Mengting Wan, and Julian McAuley. 2019. CosRec: 2D convolutional neural networks for sequential recommendation. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 2173–2176.
- [69] Yuhao Yang, Chao Huang, Lianghao Xia, Yuxuan Liang, Yanwei Yu, and Chenliang Li. 2022. Multi-Behavior Hypergraph-Enhanced Transformer for Sequential Recommendation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) (KDD '22). Association for Computing Machinery, New York, NY, USA, 2263–2274. doi:10.1145/3534678.3539342
- [70] Haochao Ying, Fuzhen Zhuang, Fuzheng Zhang, Yanchi Liu, Guandong Xu, Xing Xie, Hui Xiong, and Jian Wu. 2018. Sequential recommender system based on hierarchical attention network. In *IJCAI international joint conference on artificial intelligence*.
- [71] Fajie Yuan, Alexandros Karatzoglou, Ioannis Arapakis, Joemon M. Jose, and Xiangnan He. 2019. A Simple Convolutional Generative Network for Next Item Recommendation. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining* (Melbourne VIC, Australia) (WSDM '19). Association for Computing Machinery, New York, NY, USA, 582–590. doi:10.1145/3289600.3290975

- [72] Zheng Yuan, Fajie Yuan, Yu Song, Youhua Li, Junchen Fu, Fei Yang, Yunzhu Pan, and Yongxin Ni. 2023. Where to go next for recommender systems? id-vs. modality-based recommender models revisited. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2639–2649.
- [73] Lei Zheng, Vahid Noroozi, and Philip S. Yu. 2017. Joint Deep Modeling of Users and Items Using Reviews for Recommendation. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining* (Cambridge, United Kingdom) (*WSDM '17*). Association for Computing Machinery, New York, NY, USA, 425–434. doi:10.1145/3018661.3018665
- [74] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.